

CSCE 638 NATURAL LANGUAGE PROCESSING · MODULE 09 · LECTURE NOTES

Generation and Decoding

The model gives a distribution. You still have to choose.

MODULE OUTCOMES

- Explain the decoding strategies and their trade.
- Explain why the most likely sequence is not the best output.
- Explain the degeneration failure and its cause.
- Explain constrained and structured generation.
- Choose a strategy for a stated requirement.

1 The choice

THE MODEL GIVES $P(\text{NEXT TOKEN})$. DECODING IS A SEPARATE ALGORITHM WITH ITS OWN BEHAVIOUR.

A language model outputs a probability distribution at each step — and turning a sequence of distributions into one concrete sequence of text requires a search or sampling procedure, which is a design decision entirely independent of the model's parameters.

So two systems using the identical model can differ substantially in output quality — and any report that omits the decoding configuration is not reproducible, which makes a surprising fraction of published generation comparison ambiguous.

And the right strategy depends entirely on the task. A translation wants the single best rendering; a story wants variety — and no single setting serves both, which is §2's subject and is a real tension rather than a tuning problem.

Which makes decoding a parameter of your system rather than an implementation detail — and one of the very few places where a large quality change costs no training whatsoever, which is why §4 ends on measuring it.

GREEDY	take the argmax at every step. Deterministic, fast, and myopic -- one early mistake is unrecoverable.
BEAM SEARCH	keep the k best partial sequences. Better for tasks with one right answer. Needs length normalisation, or it prefers short outputs (shorter sequences have higher probability).
SAMPLING	draw from the distribution. Diverse, and it occasionally draws something absurd from the long tail.
TOP-k	sample from the k most likely tokens. A fixed k is wrong when the distribution is very peaked or very flat.
TOP-p	sample from the smallest set whose cumulative probability exceeds p. Adapts to the distribution's shape, which is why it displaced top-k.
TEMPERATURE	scales the logits before the softmax. Low -> sharper, high -> flatter. Orthogonal to all of the above.

Top-p adapts to the distribution's shape — when the model is confident the set is small, and when it is uncertain the set is large — **which is exactly the property a fixed k lacks, and is the reason it won. The length-normalisation point about beam search is worth remembering:** without it, beam search systematically prefers short outputs, because every additional token multiplies the probability by something less than one, and this produces truncated translations in a way that looks like a model failure.

2 The likelihood trap

THE MOST LIKELY SEQUENCE IS REPETITIVE AND DULL, WHICH IS A REAL AND SURPRISING RESULT

Increasing the beam width past a fairly modest size makes output worse on open-ended tasks — which is exactly backwards if you believe the model's probability is a quality score, and it is a reliably reproducible finding.

And the explanation is that human text is not the maximum of the distribution it came from. People produce surprising, informative text; the mode of the distribution is bland, and the highest-probability continuations include outright degenerate loops that no human would write.

So repetition is the characteristic failure, and its mechanism is specific: once a phrase has repeated, its probability rises, which makes repeating it again more likely still — a positive feedback loop in the decoding, not in the model, which is why the fixes are all decoding-time.

Which is why sampling-based methods win for open-ended text and beam search still wins decisively for translation — because translation genuinely does have approximately one right answer, so the mode is the target and searching for it is correct.

- **Sampling with top-p**, which avoids the mode entirely — **the standard answer for open-ended text**, and the first thing to try.
- **A repetition penalty**, which reduces the probability of tokens already used — **effective and crude**, since **some repetition is correct**: a proper noun, a technical term, or a deliberate refrain.
- **Blocking repeated n-grams** outright, which is cruder still and **breaks legitimately repeated names and terms** — usable for short outputs and a poor idea for long ones.
- **And contrastive methods**, which penalise tokens whose representations are too similar to the recent context — **better quality for more computation**, and a more principled version of the same idea.
- **Note that all four are decoding-time fixes to a decoding-time problem, which is why none of them requires retraining** — and is the clearest demonstration that §1's separation is a real one. **The repetition penalty's crudeness is worth stating explicitly**, because it produces its own characteristic errors that are easy to misattribute to the model.

3 Structured output

CONSTRAINED DECODING MAKES INVALID OUTPUT IMPOSSIBLE RATHER THAN UNLIKELY

If the output must be valid JSON, must match a schema, or must be drawn from a fixed set of options, enforce that during decoding — by masking out, at each step, every token that would make the output invalid or unparseable.

Which is strictly better than generating and then validating — a retry loop is slower, is sometimes unbounded, and silently biases the output distribution toward whatever the model happens to produce validly, which is a bias nobody chose and nobody measures.

And it is precisely the same argument as CSCE 713 Module 10 §2's: make the invalid state unrepresentable rather than detecting it after the fact — and CSCE 713 Module 04 §4's parse-don't-validate, arriving in a generation setting. Three courses, one principle.

With one caveat worth measuring rather than assuming away: constraining the output can degrade content quality, because the model is being pushed off its preferred distribution and may have no good continuation within the permitted set — so compare against the unconstrained version on content, not only on validity.

4 Choosing, and measuring

The requirement	Use	Why
One correct answer exists	Beam search, with length normalisation.	The mode is the target — translation, parsing, structured extraction.
Open-ended text	Top-p sampling.	The mode is degenerate (§2), so searching for it actively hurts.
Determinism required	Greedy, or temperature 0.	Reproducibility, at a real quality cost — and worth it when the output feeds another system.
Output must parse	Constrained decoding.	Invalid output made impossible (§3), rather than merely unlikely.
Diversity wanted	Higher temperature, or higher p.	And measure the quality cost, which is real and non-linear.

Report your decoding configuration with every result — strategy, temperature, p or k, length penalty, and repetition penalty. It is as much a part of the system as the model weights are, and omitting it makes a number unreproducible in a way that is entirely avoidable. Module 13 §2 treats this as part of an honest claim.

AND THE THING TO MEASURE

Sweep temperature and p against your actual task metric — the curve is usually non-monotonic with a fairly clear best region, and finding it costs an hour and no training at all.

Measure variance, not only the mean. A sampling strategy produces a *distribution of outputs*, so reporting one sample's score is reporting a draw rather than a system — run it several times and report the spread (Module 10 §1).

And measure the failure modes separately — repetition, premature truncation, and invalid format are different failures with different causes and different fixes, and an aggregate quality score hides all three.

Which is the cheapest available quality improvement in this entire course — no training, no data, no new model — and frequently a larger effect than upgrading to a bigger model would produce, which makes skipping it an expensive habit.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Holtzman et al. — The Curious Case of Neural Text Degeneration (free)	§2 in the original, and the paper that introduced top- p — the human-text-is-not-the-mode argument is made with data.
Jurafsky & Martin, the decoding sections (free)	§1's strategies, with beam search and length normalisation done carefully.
Welleck et al. — Neural Text Degeneration with Unlikelihood Training (free)	§2's repetition problem attacked at training time instead — a useful contrast with the decoding-time fixes.
The guidance and outlines libraries (free)	§3 as working code — grammar-constrained decoding you can use today.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Generate from one model with greedy, beam, and top- p** and compare the outputs.
- 2 **Increase the beam width** on an open-ended prompt and observe the quality.
- 3 **Remove length normalisation** from beam search and report what happens to the output length.
- 4 **Produce a repetition loop** and trace the probability of the repeated phrase as it goes.
- 5 **Fix it four ways** and compare the side effects of each.

- 6 **Show the repetition penalty breaking a repeated proper noun.**
- 7 **Implement constrained decoding** for a small JSON schema.
- 8 **Compare constrained against generate-and-retry** on latency and on content quality.
- 9 **Sweep temperature against your task metric** and find the best region.
- 10 **Report the variance** across ten samples at your chosen setting.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is decoding separate from the model, and what follows for reporting?
- 2 Name six strategies and what each does.
- 3 Why did top-p displace top-k?
- 4 Why does beam search prefer short outputs without normalisation?
- 5 Why is the most likely sequence bad for open-ended text?
- 6 Give the mechanism of the repetition loop.
- 7 Why does beam search still win for translation?
- 8 Give four repetition fixes and the cost of the crudest.
- 9 Why is constrained decoding better than generate-and-validate, and what is the caveat?
- 10 What three things should you measure?